

LAB GUIDELINES

Lab 16 — Scripting Basics for Server Administration

Maps to CompTIA Server+ objective **2.6** — Script types (Bash, Batch, PowerShell, VBS), environment variables, comment syntax, basic constructs (loops, variables, conditionals, comparators), basic data types (integers, strings, arrays), common admin tasks (startup, shutdown, service, login, account creation, bootstrap).

Run all commands on <https://killercode.com/playgrounds/scenario/ubuntu>

Required software (free, built-in): bash, coreutils

Step 1 — Script types reference (memorise for the exam)

Type	OS	Extension	Shebang / launcher
Bash	Linux/Unix	<code>.sh</code>	<code>#!/usr/bin/env bash</code>
Batch	Windows	<code>.bat</code> / <code>.cmd</code>	<code>cmd.exe</code>
PowerShell	Windows	<code>.ps1</code>	<code>#Requires -Version 5</code> / <code>pwsh</code>
VBScript	Windows (legacy)	<code>.vbs</code>	<code>cscript.exe</code>

Server+ asks you to know which language fits which platform. Linux automation = bash. Modern Windows automation = PowerShell.

Step 2 — Variables, comments, environment variables

```
mkdir -p /srv/scripts && cd /srv/scripts
cat > 01-basics.sh <<'EOF'
#!/usr/bin/env bash
# This line is a comment (Bash uses #)
NAME="srv01"                                # local variable
export ADMIN_EMAIL="ops@lab.local"          # environment variable (child procs inherit)
echo "Hostname : $NAME"
echo "Admin    : $ADMIN_EMAIL"
echo "Home env : $HOME"
echo "Path env : $PATH" | head -c 80; echo
EOF
chmod +x 01-basics.sh && ./01-basics.sh
```

Step 3 — Data types: integers, strings, arrays

```
cat > 02-types.sh <<'EOF'
#!/usr/bin/env bash
# Integer
COUNT=5
COUNT=$((COUNT + 1))
echo "count=$COUNT"

# String
GREETING="hello server"
echo "${GREETING^^}"      # to upper
echo "${#GREETING}"       # length

# Array
SERVICES=(nginx mariadb sshd)
for s in "${SERVICES[@]}"; do
    echo "service: $s"
done
```

```
echo "third: ${SERVICES[2]}"
EOF
chmod +x 02-types.sh && ./02-types.sh
```

Step 4 — Conditionals and comparators

```
cat > 03-conditionals.sh <<'EOF'
#!/usr/bin/env bash
DISK_USED=$(df --output=pcent / | tail -1 | tr -dc 0-9)
if [ "$DISK_USED" -ge 90 ]; then echo "CRIT: disk $DISK_USED%"
elif [ "$DISK_USED" -ge 75 ]; then echo "WARN: disk $DISK_USED%"
else
    echo "OK: disk $DISK_USED%"
fi
EOF
chmod +x 03-conditionals.sh && ./03-conditionals.sh
```

Comparators table the exam expects:

Numeric	String	Meaning
`-eq`	`==`	equal
`-ne`	`!=`	not equal
`-lt`	`<`	less than
`-le`	`<=`	less or equal
`-gt`	`>`	greater than
`-ge`	`>=`	greater or equal

Step 5 — Loops

```
cat > 04-loops.sh <<'EOF'
#!/usr/bin/env bash
# for loop over a list
for u in alice bob carol; do
    echo "would-create user: $u"
done

# while loop reading lines
seq 1 3 | while read n; do echo "iter $n"; done
EOF
chmod +x 04-loops.sh && ./04-loops.sh
```

Step 6 — Common admin task: startup / service management script

```
cat > 05-services.sh <<'EOF'
#!/usr/bin/env bash
SERVICES=(nginx mariadb ssh)
for svc in "${SERVICES[@]}"; do
    status=$(systemctl is-active "$svc" 2>/dev/null)
    echo "$svc: $status"
    [ "$status" != "active" ] && systemctl start "$svc"
done
EOF
chmod +x 05-services.sh && ./05-services.sh
```

Step 7 — Common admin task: account creation in bulk

```
cat > 06-accounts.sh <<'EOF'
#!/usr/bin/env bash
while IFS=, read -r user fullname group; do
    [ -z "$user" ] && continue
```

```

        id "$user" >/dev/null 2>&1 || useradd -m -c "$fullname" -G "$group" "$user"
        echo "ensured user: $user (group=$group)"
    done <<CSV
    alice,Alice Admin,wheel
    bob,Bob Operator,users
    carol,Carol DBA,users
    CSV
    getent passwd alice bob carol
    EOF
    chmod +x 06-accounts.sh && ./06-accounts.sh

```

Step 8 — Common admin task: shutdown / bootstrap

```

cat > 07-bootstrap.sh <<'EOF'
#!/usr/bin/env bash
# A fresh-host bootstrap pattern: idempotent, exits non-zero on failure
set -euo pipefail
PKGS=(curl vim htop chrony rsync)
apt update
DEBIAN_FRONTEND=noninteractive apt install -y "${PKGS[@]}"
systemctl enable --now chrony
timedatectl set-timezone UTC
echo "bootstrap complete on $(hostname -f) at $(date -Is)"
EOF
chmod +x 07-bootstrap.sh
echo "(would run on a fresh host)"

```

`set -euo pipefail` is the production-script discipline: stop on errors, undefined variables, and failed pipe segments.

Step 9 — Logging in to a server (login script)

`/etc/profile.d/00-banner.sh` is read on every interactive login:

```

cat > /etc/profile.d/00-banner.sh <<'EOF'
echo "[$(date -Is)] login: $USER from $(who am i | awk '{print $5}')" on $(hostname -f)" \
    | tee -a /var/log/logins.log >/dev/null
EOF

```

Free online tools

- **ShellCheck** (free linter)** — <https://www.shellcheck.net/>
- **Bash reference manual**** — <https://www.gnu.org/software/bash/manual/>
- **PowerShell docs**** — <https://learn.microsoft.com/en-us/powershell/>

Ready-to-run scripts (assets)

Two complete, commented scripts are in `[scripts/](scripts/)` — study them for variables, loops, conditionals, functions and traps, then run them on the **Killercoda Ubuntu** playground:

- `[scripts/server-healthcheck.sh](scripts/server-healthcheck.sh)` — a threshold-based disk / load / service health check.

```
bash scripts/server-healthcheck.sh
```

- `[scripts/backup-rotate.sh](scripts/backup-rotate.sh)` — a daily tar backup with retention (pairs with Lab 26).

```
bash scripts/backup-rotate.sh /etc /root/backups 7
```

What you learned

- The four script types Server+ tests and the platform each fits.
- Variables, environment exports, comments, integers, strings, arrays.

- Conditionals + the full comparator table.
- For and while loops.
- Real-world admin tasks: service status, bulk account creation, host bootstrap, login banner.
- `set -euo pipefail` discipline for production scripts.